

KolamKala

A full-stack web application for Kolam art analysis and pattern generation
using Computer Vision and geometric algorithms.

Final Year BCA Project | Python | FastAPI | OpenCV | Vanilla JavaScript

1. Project Overview

KolamKala is a full-stack web application built as a Final Year BCA project at **Ajeenkya DY Patil University, Pune**, under the guidance of **Prof. Sandeep Kulkarni**. It bridges traditional South Indian Kolam art with computational image analysis and geometric pattern generation.

The application allows users to upload photographs of Kolam designs and receive a structured analysis — symmetry detection, dot/loop/line counting, complexity scoring, and pattern classification. Users can also generate Kolam patterns interactively through a browser-based interface backed by a Python REST API.

Note: This project uses OpenCV-based image processing and rule-based geometric algorithms. It does not use machine learning models or neural networks.

2. Features

- Upload a Kolam image and receive a detailed structural analysis
- Detect horizontal, vertical, and rotational symmetry in patterns
- Count dots (Pulli), loops (Sikku curves), and lines (Kambi strokes)
- Calculate a complexity score using a weighted geometric formula
- Classify the uploaded Kolam into one of four standard types
- Dynamically generate Kolam patterns with adjustable parameters
- Interactive frontend built with plain HTML, CSS, and JavaScript
- REST API with full Swagger / OpenAPI documentation at /docs

3. Tech Stack

Frontend

Technology	Purpose
HTML5	Page structure and markup
CSS3	Base styling
Tailwind CSS (CDN)	Utility-first responsive design
Vanilla JavaScript	DOM manipulation, API calls, canvas rendering

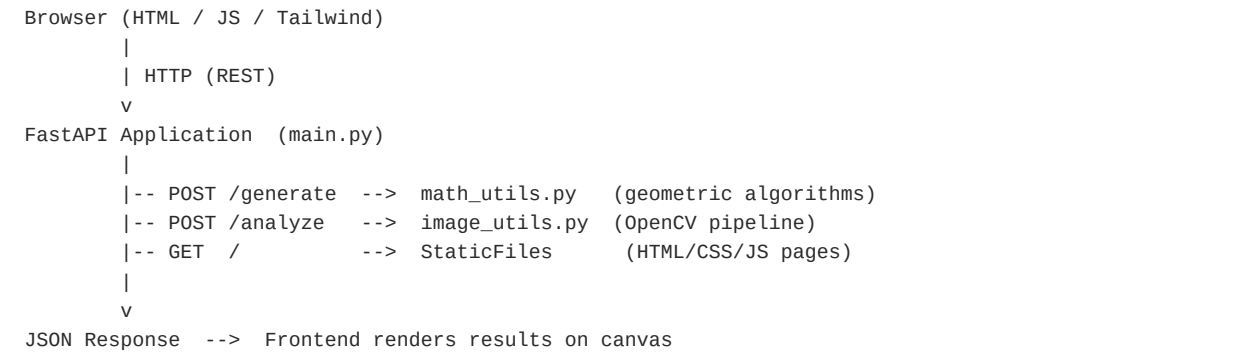
Backend

Technology	Purpose
Python 3.11	Core application language
FastAPI	REST API framework with async support
Uvicorn	ASGI server for running the application

Computer Vision

Library	Purpose
OpenCV	Image preprocessing, contour detection, symmetry analysis
NumPy	Array operations and numerical computation
Pillow	Image format handling and preprocessing

4. Project Architecture



The FastAPI server acts as both the API backend and the static file server. API routes are registered before the static file mount so they always take priority.

5. How the Analyzer Works

The image analysis pipeline processes an uploaded Kolam image through these steps:

Step 1 — Preprocessing

Convert to grayscale, apply Gaussian blur, then binarize using adaptive thresholding to isolate pattern elements from the background.

Step 2 — Dot Detection

Apply Hough Circle Transform, filter by radius range, and count remaining circles as Pulli (dot) count. Skin mask check skips when skin tones cover more than 40% of the image.

Step 3 — Loop Detection

Run Canny edge detection, find contours, then filter by area and circularity ratio to identify closed curve structures (Sikku loops).

Step 4 — Line Detection

Apply Probabilistic Hough Line Transform, filter by minimum segment length, count qualifying segments as Kambi strokes.

Step 5 — Symmetry Analysis

Flip horizontally/vertically and compute pixel-wise difference. Rotate 180° for rotational symmetry. Confirmed when mean difference falls below threshold.

Step 6 — Complexity Scoring

$$\text{score} = (\text{dot_count} \times 0.5) + (\text{loop_count} \times 2.0) + (\text{line_count} \times 1.0)$$

Step 7 — Classification

Apply rule-based classifier using counts and symmetry results (see Section 6).

Complexity Score Ranges

Score Range	Complexity Label
0 – 14	Simple
15 – 39	Moderate

Score Range	Complexity Label
40 – 89	Complex
90 and above	Highly Complex

6. Pattern Classification Logic

Classification is rule-based, evaluated in the following priority order:

Condition	Classification
Rotational symmetry detected AND (loops >= 6 OR dots >= 16)	Mandala
loop_count >= 8	Sikku
line_count >= 15	Kambi
None of the above	Pulli

This approach reflects the structural characteristics of each Kolam type without requiring a trained model.

7. Pattern Generator Logic

Type	Description
Basic (Pulli)	Grid of dots placed at regular intervals with configurable spacing and size
Symmetric	Dot grid with reflected symmetry axes drawn programmatically
Diagonal	Dots placed along diagonal axes with angular offsets
Sikku	Closed loop curves generated around dot anchor points using parametric equations

The size parameter controls grid density. Returns dot coordinates, line segments, and loop path data as JSON rendered on an HTML5 Canvas.

8. API Endpoints

Base URL: <http://localhost:8090> Docs: <http://localhost:8090/docs>

GET /health

Returns service status and available endpoint list.

POST /generate

Generates a Kolam pattern based on type and grid size.

Field	Type	Allowed Values
type	string	"basic", "symmetric", "diagonal", "sikku"
size	integer	Grid dimension, e.g. 3 to 10

POST /analyze

Analyzes an uploaded Kolam image. Request is multipart/form-data with field 'file' (JPEG or PNG).

```
{
  "pattern_type": "Pulli",
  "symmetry": { "horizontal": true, "vertical": true, "rotational": false },
  "complexity": "Moderate", "complexity_score": 28.5,
  "dot_count": 21, "loop_count": 3, "line_count": 9
}
```

9. Installation and Setup

Prerequisites: Python 3.11+, pip, a modern web browser.

```
pip install fastapi uvicorn opencv-python numpy pillow python-multipart
```

10. Running Locally

```
git clone https://github.com/kolambepalak/kolam-kala.git
cd kolam-kala
pip install fastapi uvicorn opencv-python numpy pillow python-multipart
uvicorn backend.main:app --host 0.0.0.0 --port 8090 --reload
```

Open your browser at **http://localhost:8090**.

11. Project Structure

```
KolamKala/
|-- backend/
|   |-- main.py           # FastAPI app: routes, CORS, static mount
|   |-- utils/
|       |-- image_utils.py # OpenCV analysis pipeline
|       |-- math_utils.py  # Geometric pattern generation
|-- artifacts/api-server/frontend/
|   |-- index.html |-- generator.html |-- analyzer.html
|   |-- history.html |-- contact.html
|   |-- app.js   |-- styles.css
|-- documentation/
|   |-- KolamKala_Blackbook.pdf
|   |-- KolamKala_TechDoc.pdf
|   |-- KolamKala_Presentation.pptx
|-- README.md
```

12. Screenshots

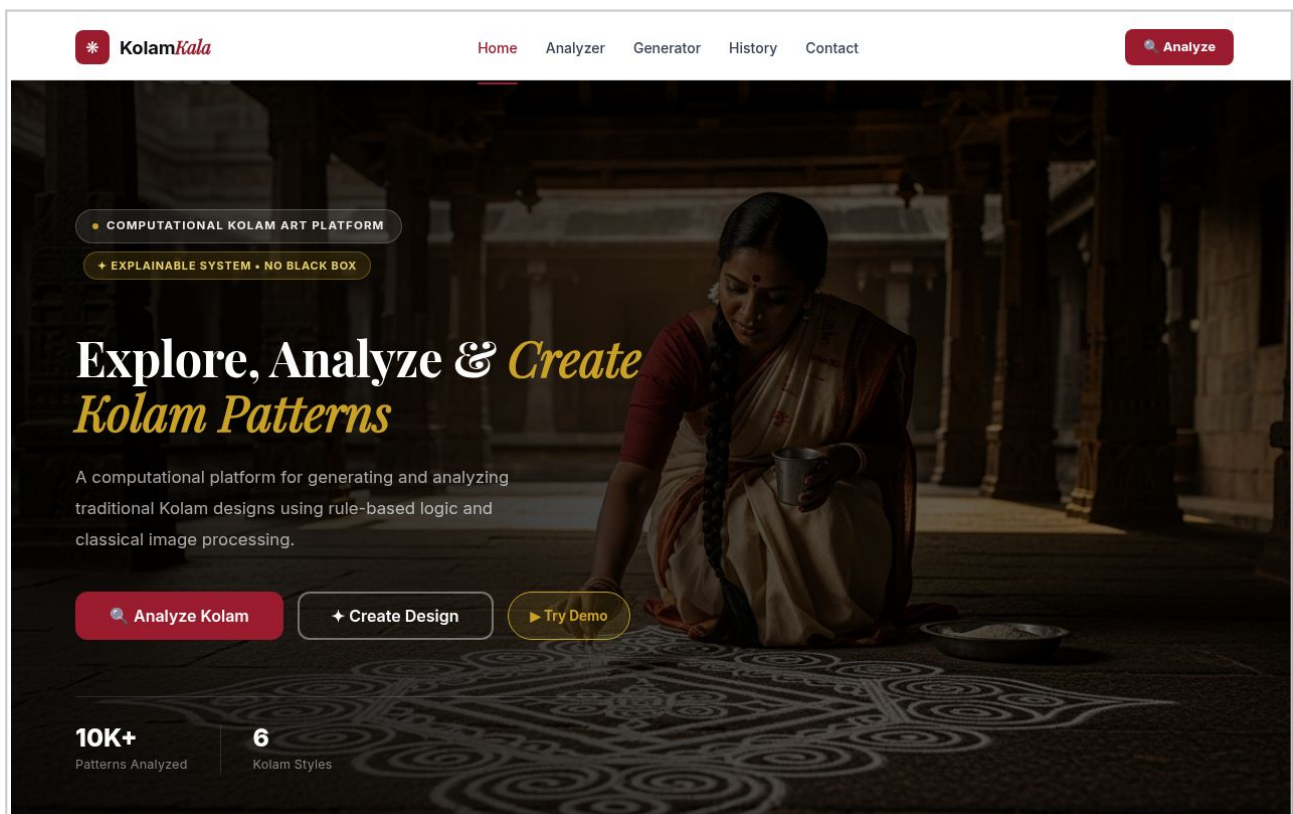


Fig 1 — Home Page: Hero section with navigation and call-to-action

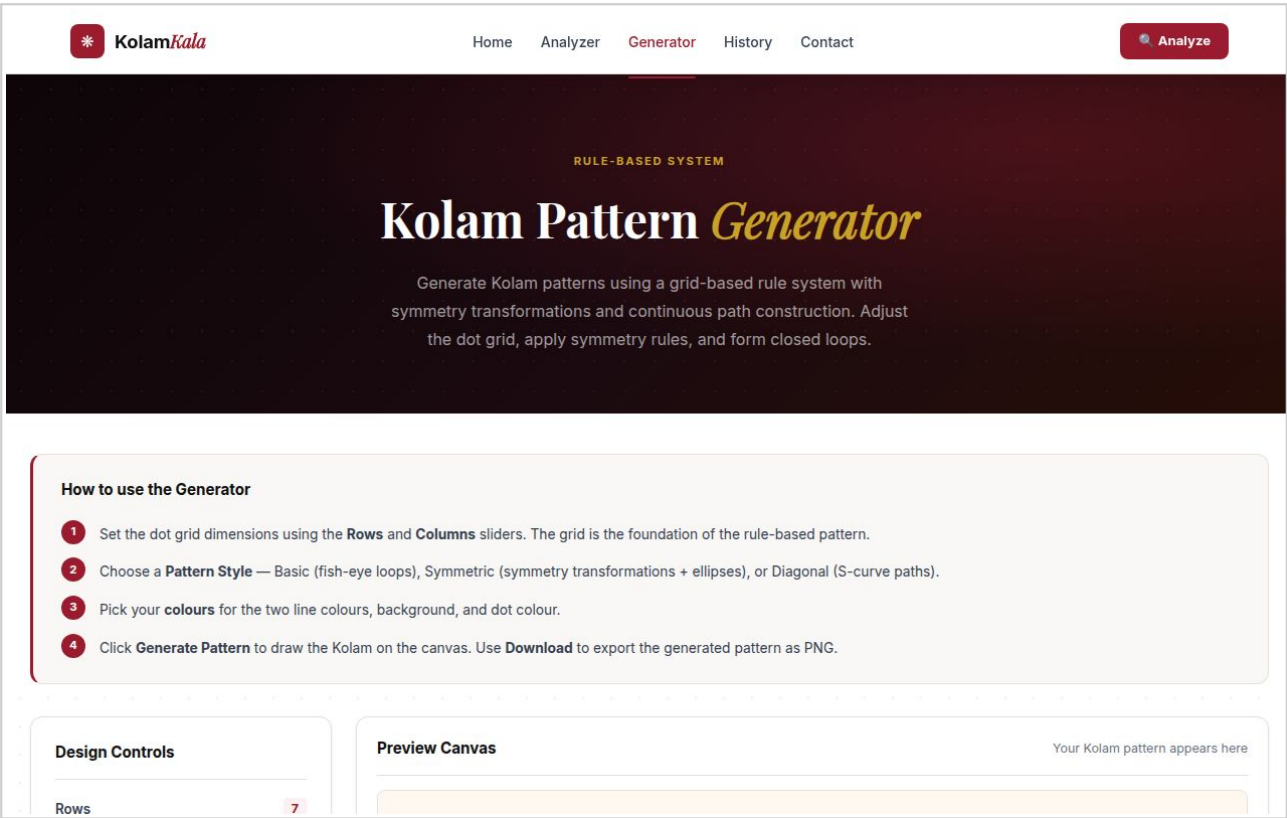


Fig 2 — Generator Page: Pattern design controls and canvas preview

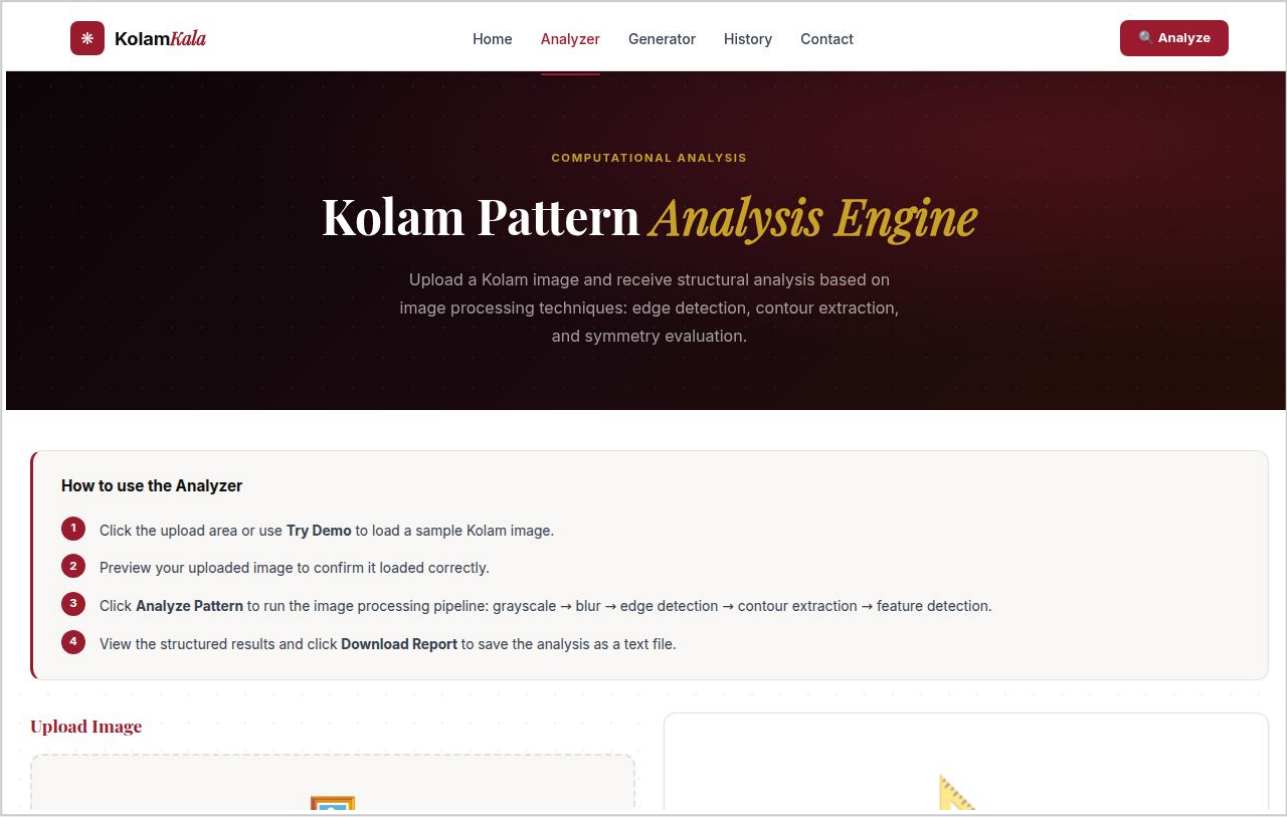


Fig 3 — Analyzer Page: Image upload and analysis engine interface

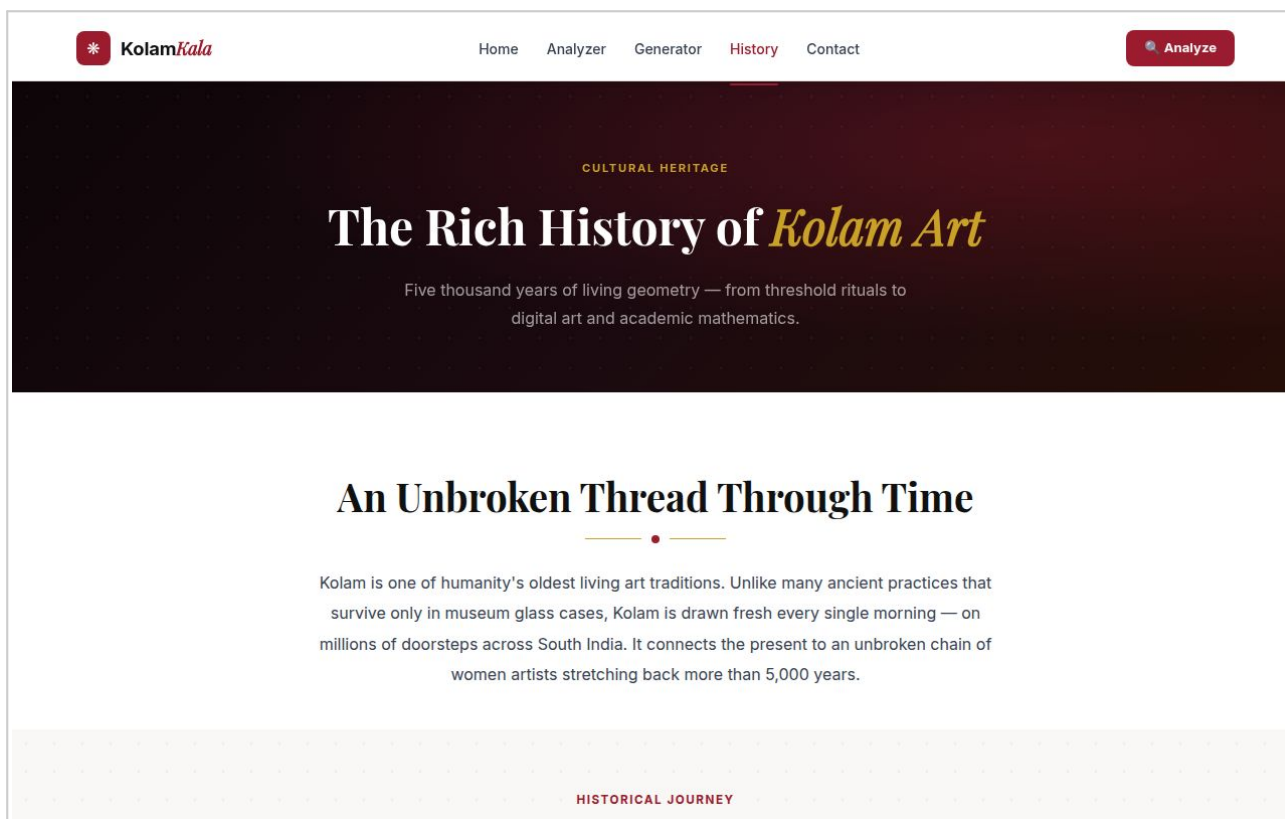


Fig 4 — History Page: Cultural context and pattern history

13. Academic Documentation

Document	Description
KolamKala_Blackbook.pdf	43-page BCA blackbook covering project introduction, literature review, system design, implementation, testing, and conclusion
KolamKala_TechDoc.pdf	19-page technical reference covering API contracts, CV pipeline, algorithms, and deployment notes
KolamKala_Presentation.pptx	17-slide viva presentation covering all project phases

Documents are available in the `documentation/` folder.

14. Future Improvements

- Add a trained classifier (CNN or SVM) for improved accuracy beyond rule-based thresholds
- Implement user accounts for saving and revisiting patterns
- Support batch upload for multi-Kolam image analysis
- Add export feature to download patterns as SVG or PNG
- Build a curated Kolam dataset for future ML research
- Improve dot detection for complex or overlapping patterns

15. Limitations

- Accuracy depends on image quality — blurry or poorly lit photos may yield incorrect counts
- Skin mask logic may reduce detection accuracy when skin tones overlap the pattern
- Rule-based classification may misclassify unusual or mixed-style Kolams
- Symmetry detection may be affected by perspective distortion in real photographs

- Generator produces simplified geometric representations, not hand-drawn curves
- No database — results are not persisted between sessions

16. Author

Palak Kolambe

BCA Student | AI & ML Enthusiast | Python Developer

Detail	Information
University	Ajeenkya DY Patil University, Pune
Program	Bachelor of Computer Applications (BCA) — Final Year
Project Guide	Prof. Sandeep Kulkarni

17. License

This project is licensed under the MIT License. You are free to use, modify, and distribute this project with proper attribution.

KolamKala — Bridging traditional Indian art with computational geometry.